

CSCI 12532

Computer Architecture and Design

Lecture 02

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Contents

1. History of Computers
2. Pillars of Computer Architecture
3. CPU Structure and Types of CPU Architectures
4. Flynn's Taxonomy of Computer Architecture

History of Computers

Computers have come a long way from their creation. This first computer was created in 1822 by Charles Babbage and was created with a series of vacuum tubes, ultimately weighing 700 pounds- far larger than computers today. Computers have vastly decreased in size due to the invention of the transistor in 1947, which revolutionized computing by replacing bulky vacuum tubes with smaller components that made computers more compact and also more reliable. This led to an era of rapid technological advancement, leading to the development of integrated circuits, microprocessors, and eventually the smaller, lighter personal computers that have become indispensable in modern society.

As the world of computers keeps progressing, there is a general concept of making them smaller, but at the same time seeing a generational step in improvement. The improvements allow the daily tasks of many users to become shorter, making their tasks quicker and easier to accomplish.

- First Generation: Vacuum Tubes (1946–1959)
- Second Generation: Transistors (1959–1965)
- Third Generation: Integrated Circuits (1965–1971)
- Fourth Generation: Microprocessors (1971–Present)
- Fifth Generation: Artificial Intelligence (Present/Future)

Let's refresh your memory on the Computer timeline

1. What technological development defines the First Generation of computers?
2. While transistors were invented in 1947, how long did it take for them to be used in computer hardware?
3. What key technological advancement marked the Third Generation of computers?
4. What was the name of the chip, developed in 1971, that included all the components of a computer on a single silicon chip?
5. How did the shift from 3rd generation to microprocessors change the scale and accessibility of computing?

[History of the Computer Timeline](#)

The Analytical Engine

As Babbage built prototypes of his Difference Engine, he began to envision a much more powerful computing device he called the Analytical Engine.

Babbage's initial notes on the Analytical Engine appear in 1837, but the most complete description appears in a 1842 paper by Luigi Federico Menabrea, who was reporting on a lecture Babbage gave in 1840. Ada Lovelace translated Menabrea's paper from French into English and provided notes that were three times longer than the original.

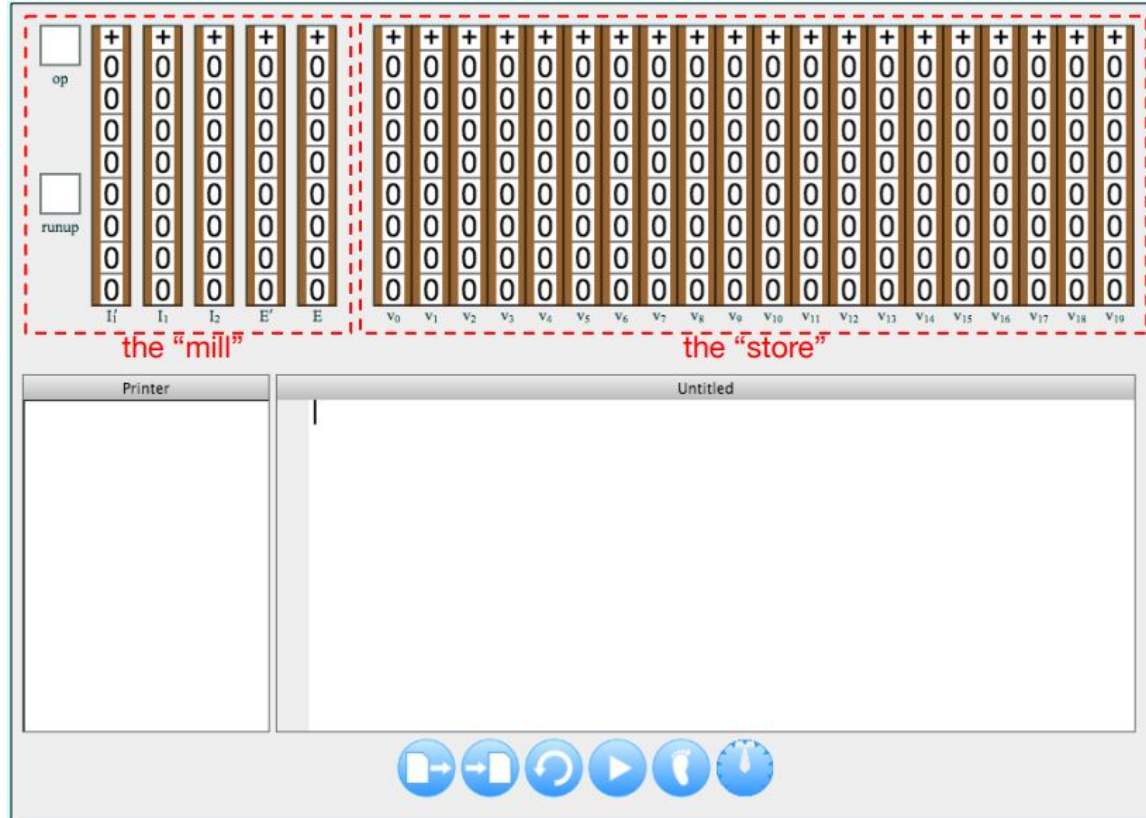
The essential difference between the Difference Engine and the Analytical Engine is that the Analytical Engine was designed to be programmable, allowing users to perform any sequence of calculations. The programs were encoded on punched cards in the manner of the Jacquard loom.



The Jacquard Loom

[How was it Made? Jacquard weaving](#)

Structure of the Analytical Engine



The Store

- Each column holds a single integer as in the Difference Engine.
- Numbers in the Analytical Engine are signed.
- Each column has a numeric address: v_0 , v_1 , v_2 , v_3 , and so on.

The Mill

- The op indicator holds the current operation (+, −, ×, ÷)
- The mill has five columns
 - I_1 and I_2 are the input values
 - E is the output value
 - I_1' and E' are used to store extra digits for the × and ÷ operations
- The runup indicator is set when the result of an operation changes sign.

Ada Byron, The First Programmer

Ada was convinced of the potential of Babbage's Analytical Engine and wrote extensive notes on its design, along with several complex mathematical programs that have led many people to characterize her as the first programmer. In 1980, the U.S. Department of Defense named the programming language Ada in her honor.

She saw the machine as more than just a tool for numerical calculations, envisioning its use for general-purpose computing.

Key Contributions

- Published extensive notes on the Analytical Engine, explaining its functionality and potential applications.
- Included an algorithm for calculating Bernoulli numbers, which is considered the first computer program in history.
- Her work demonstrated the machine's ability to perform complex tasks beyond arithmetic, such as processing symbols and creating music or art.

The Turing machine

Turing looked at computers of his day and asked “What are these people doing?” He thought about it and realized these folks would write something down, think for a bit, write something else or erase something, think some more, write or erase, and so on. They would move from mental state to mental state as they worked, deciding what to do next based on what mental state they were in and what was currently written.

He realized he could capture this process formally and keep the model very simple: instead of a two-dimensional page, a single one-dimensional tape would do, and instead of randomly jumping around, it would be okay to move one cell at a time.

[Turing Machine Examples](#)

A Turing machine is a mathematical abstraction of computation first introduced by Alan M. Turing in 1936. It consists of an infinite tape divided into discrete cells, a tape-head that can read and write symbols on the tape, and a finite set of internal states with specified transition rules. Turing machines operate in discrete steps, changing state and tape content based solely on the current state and scanned symbol.

In practical terms, Turing's abstraction illuminates the essence of real computers. All modern computers share the basic structure of the Turing tape–head–control paradigm: they have memory (analogous to the tape), a processing unit (analogous to the finite-state control), and input/output devices. Yet real computers differ from the ideal Turing machine (for instance, having finite memory and random access rather than an infinite linear tape). Even so, the Turing machine remains the theoretical touchstone. As one survey notes, “the universal machine is today by many still considered as the model for the modern computer”

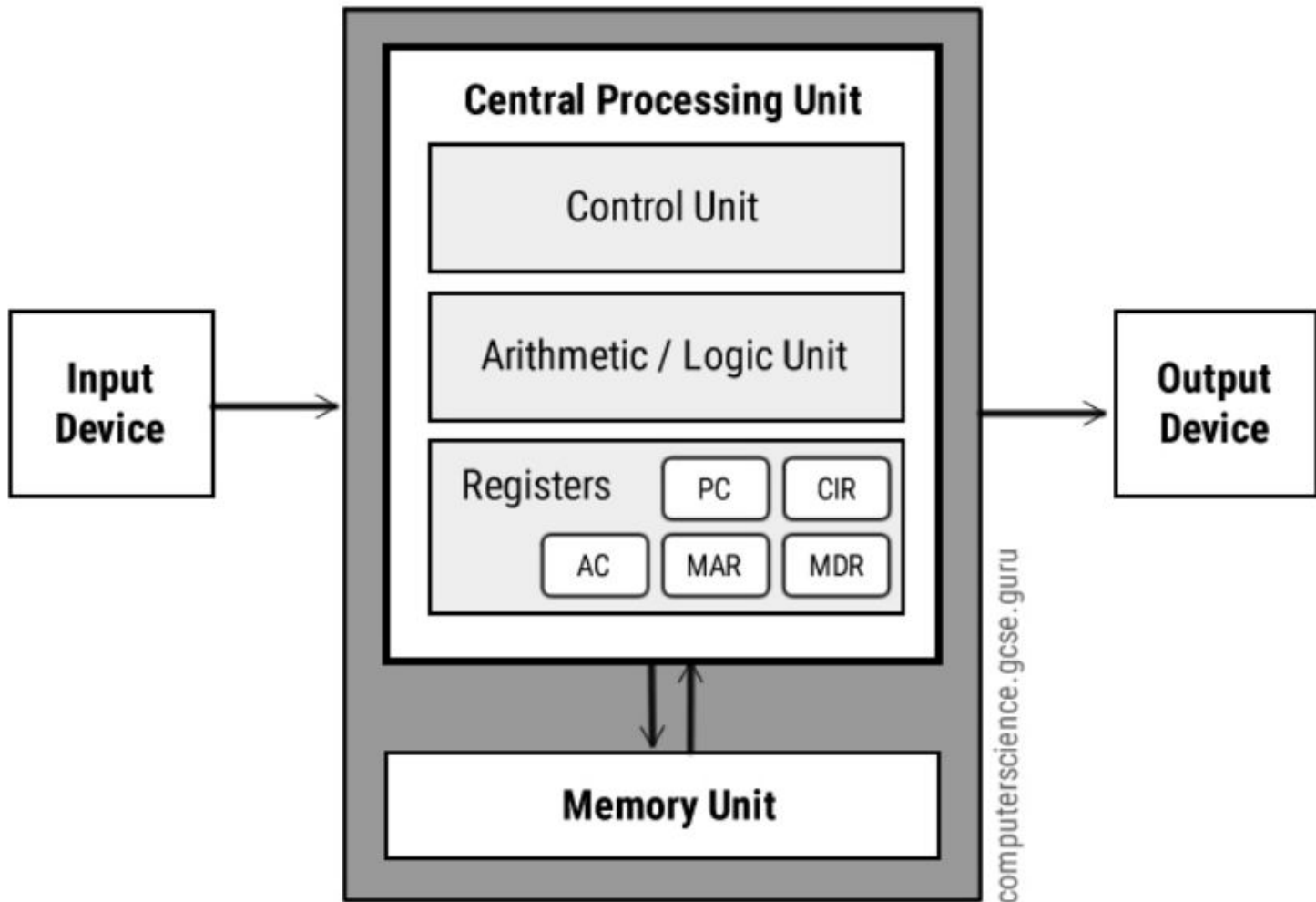
The Von Neumann Architecture

Von Neumann architecture was first published by John von Neumann in 1945.

His computer architecture design consists of a Control Unit, Arithmetic and Logic Unit (ALU), Memory Unit, Registers and Inputs/Outputs.

Von Neumann architecture is based on the stored-program computer concept, where instruction data and program data are stored in the same memory. This design is still used in most computers produced today.

All modern processors, regardless of their ISA, adhere to the von Neumann architecture model. The general-purpose design of the von Neumann architecture allows it to execute any type of program. It uses a stored-program model, meaning that the program instructions reside in computer memory along with program data, and both are inputs to the processor.



The von Neumann architecture serves as the foundation for most modern computers. The von Neumann architecture consists of five main components:

- The processing unit executes program instructions.
- The control unit drives program instruction execution on the processing unit. Together, the processing and control units make up the CPU.
- The memory unit stores program data and instructions.
- The input unit(s) load program data and instructions on the computer and initiate program execution.
- The output unit(s) store or receive program results.

Buses connect the units, and are used by the units to send control and data information to one another. A bus is a communication channel that transfers binary values between communication endpoints (the senders and receivers of the values). For example, a data bus that connects the memory unit and the CPU could be implemented as 32 parallel wires that together transfer a 4-byte value, 1-bit transferred on each wire. Typically, architectures have separate buses for sending data, memory addresses, and control between units. The units use the control bus to send control signals that request or notify other units of actions, the address bus to send the memory address of a read or write request to the memory unit, and the data bus to transfer data between units.

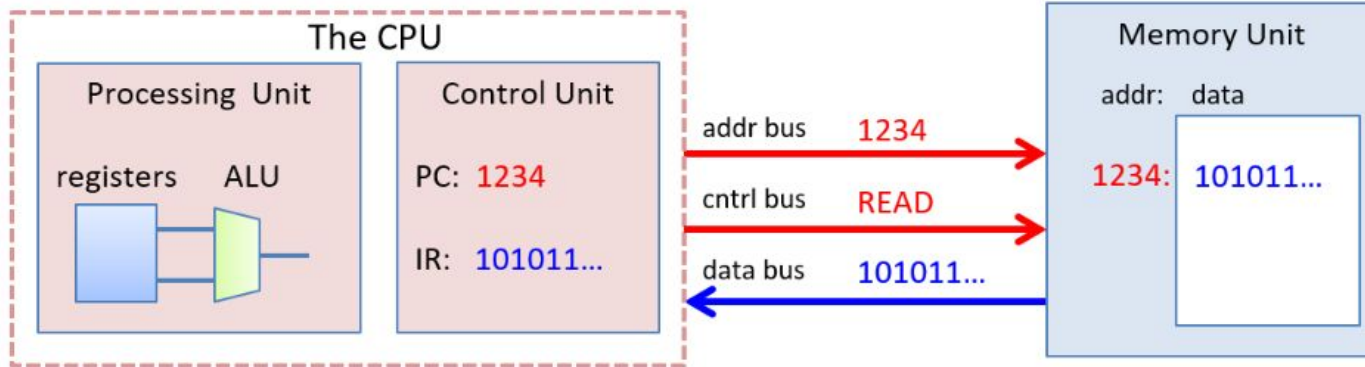
The von Neumann Machine in Action: Executing a Program

The five units that make up the von Neumann architecture work together to implement a fetch-decode-execute-store cycle of actions that together execute program instructions. This cycle starts with a program's first instruction, and is repeated until the program exits:

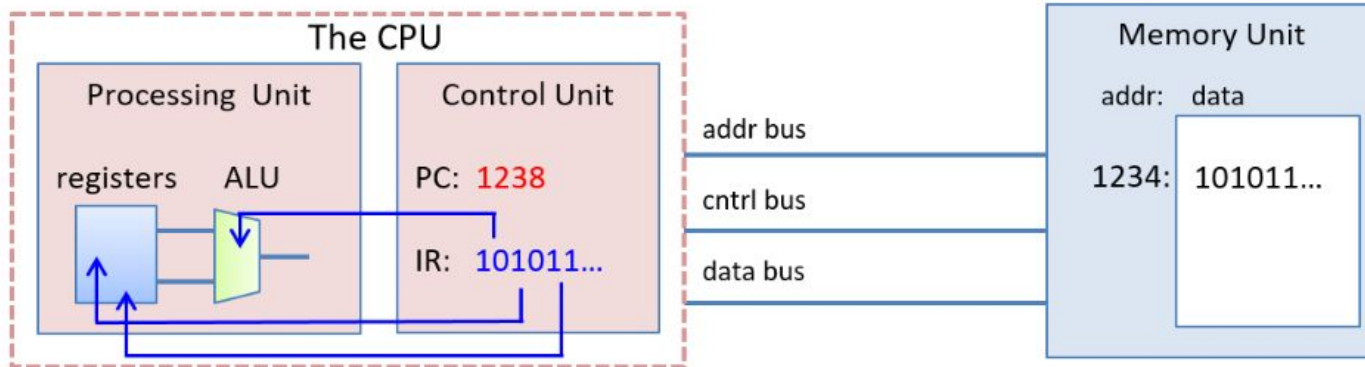
1. The control unit fetches the next instruction from memory. The control unit has a special register, the program counter (PC), that contains the address of the next instruction to fetch. It places that address on the address bus and places a read command on the control bus to the memory unit. The memory unit then reads the bytes stored at the specified address and sends them to the control unit on the data bus. The instruction register (IR) stores the bytes of the instruction received from the memory unit. The control unit also increments the PC's value to store the address of the new next instruction to fetch.

2. The control unit decodes the instruction stored in the IR. It decodes the instruction bits that encode which operation to perform and the bits that encode where the operands are located. The instruction bits are decoded based on the ISA's definition of the encoding of its instructions. The control unit also fetches the data operand values from their locations (from CPU registers, memory, or encoded in the instruction bits), as input to the processing unit.
3. The processing unit executes the instruction. The ALU performs the instruction operation on instruction data operands.
4. The control unit stores the result to memory. The result of the processing unit's execution of the instruction is stored to memory. The control unit writes the result to memory by placing the result value on the data bus, placing the address of the storage location on the address bus, and placing a write command on the control bus. When received, the memory unit writes the value to memory at the specified address.

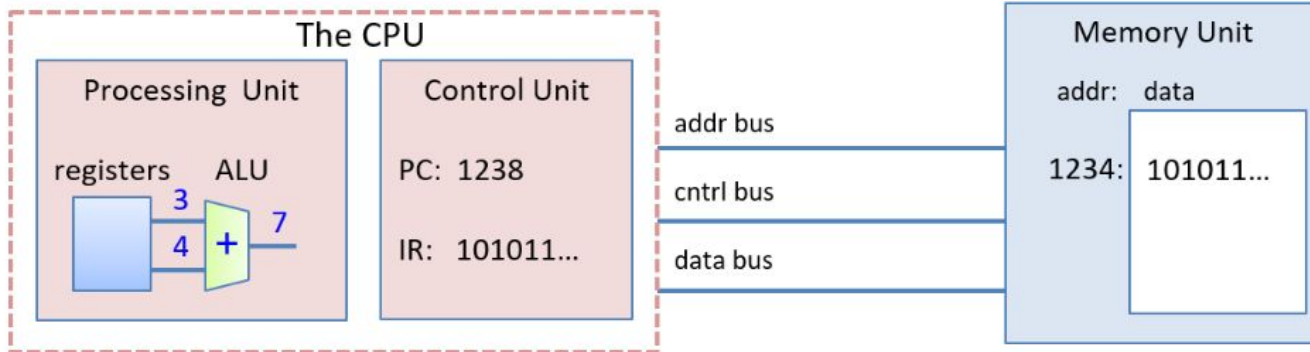
The input and output units are not directly involved in the execution of program instructions. Instead, they participate in the program's execution by loading a program's instructions and data and by storing or displaying the results of the program's computation.



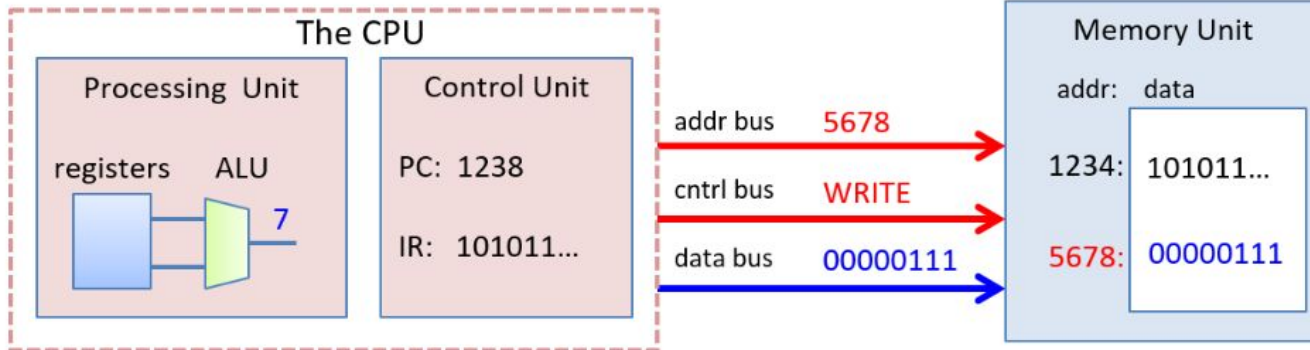
1. Fetch: Read instruction bits from memory at address in PC (1234), and store in IR



2. Decode: instruction bits in IR encode which registers store operands & the ALU operation



3. Execute: ALU performs instruction operation (+) on operands (3,4) to compute result (7)



4. Store: the control unit stores the ALU result (7, binary 00000111) to memory

Advantages of the Von Neumann Architecture

1. Simplicity

Von Neumann architecture is relatively simple to understand and implement.

2. Flexibility

The ability to store instructions in memory allows for the execution of different programs without changing the hardware.

3. Efficiency

The nature of processing instructions maximizes the utilization of the CPU

Limitations of Von Neumann Architecture

1. Von Neumann Bottleneck

The sequential processing model can limit the overall performance of the system.

2. Memory Bottleneck

The CPU and memory access share the same bus, leading to potential performance limitations.

3. Lack of Parallelism

Von Neumann architecture does not inherently support parallel processing.

Is Von Neumann architecture impeding the power of AI computing?

Modern computer processors are quite efficient at performing the discrete computations they're usually tasked with. Though their efficiency nosedives when they must wait for data to move back and forth between memory and compute, they're designed to quickly switch over to work on some unrelated task. But for AI computing, almost all the tasks are interrelated, so there often isn't much other work that can be done when the processor gets stuck waiting, said IBM Research scientist Geoffrey Burr.

In that scenario, processors hit what is called the von Neumann bottleneck, the lag that happens when data moves slower than computation. It's the result of von Neumann architecture, found in almost every processor over the last six decades, wherein a processor's memory and computing units are separate, connected by a bus. This setup has advantages, including flexibility, adaptability to varying workloads, and the ability to easily scale systems and upgrade components.

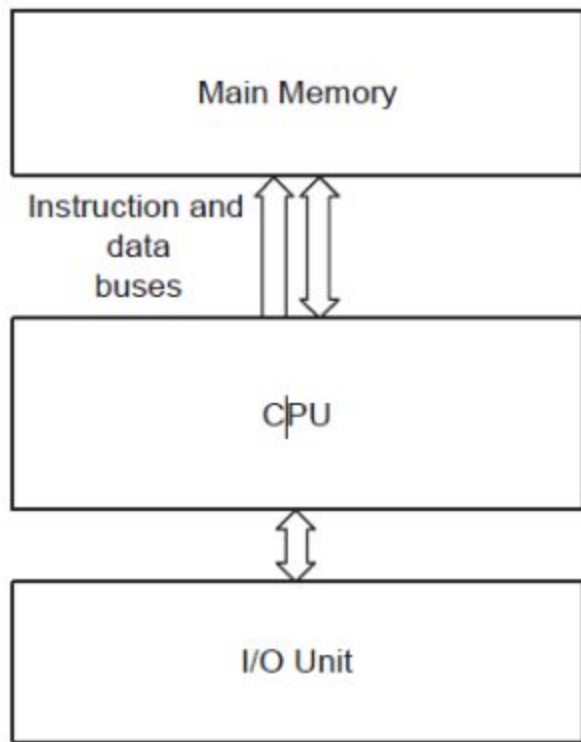
But for AI computing, whose operations are simple, numerous, and highly predictable, a conventional processor ends up working below its full capacity while it waits for model weights to be shuttled back and forth from memory.

For AI computing, the von Neumann bottleneck creates a twofold efficiency problem: the number of model parameters (or weights) to move, and how far they need to move. More model weights mean larger storage, which usually means more distant storage.

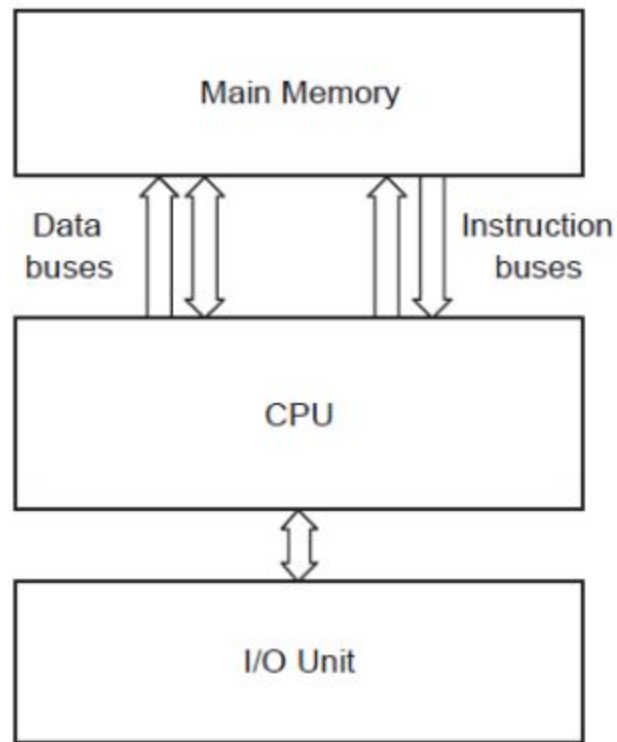
The main energy expenditure during AI runtime is spent on data transfers — bringing model weights back and forth from memory to compute. By comparison, the energy spent doing computations is low. In deep learning models, for example, the operations are almost all relatively simple matrix vector multiplication problems. Compute energy is still around 10% of modern AI workloads, so it isn't negligible.

Activity

Compare the Von Neumann Architecture with the Harvard Architecture



(a)



(b)

2. Pillars of Computer Architecture

Quick Recap from last week!

What are the three pillars of computer architecture?

Instruction set architecture (ISA)

Instruction set architecture (ISA) is a bridge between the software and hardware of a computer. It functions as a programmer's viewpoint on a machine. Computers can only comprehend binary language (0 and 1), but humans can comprehend high-level language (if-else, while, conditions, and the like). Consequently, ISA plays a crucial role in user-computer communications by translating high-level language into binary language.

In addition, ISA outlines the architecture of a computer in terms of the fundamental activities it must support. It's not involved with implementation-specific computer features.

Instruction set architecture dictates that the computer must assist:

- Arithmetic/logic instructions: These instructions execute various mathematical or logical processing elements solely on a single or maybe more operands (data inputs).
- Data transfer instructions: These instructions move commands from the memory or into the processor registers, or vice versa.
- Branch and jump instructions: These instructions are essential to interrupt the logical sequence of instructions and jump to other destinations.

Different CPUs implement different sets of instructions.

Examples: ARM, Intel x86, IBM/Motorola PowerPC(Macintosh), MIPS, Inter IA32 ...

Some things an ISA must specify:

- A way to reference registers and memory
- The computational operations available
- How to control the sequence of instructions
- A binary representation for all of the above

Microarchitecture

Microarchitecture, unlike ISA, focuses on the implementation of how instructions will be executed at a lower level. This is influenced by the microprocessor's structural design.

Microarchitecture is a technique in which the instruction set architecture incorporates a processor. Engineering specialists and hardware scientists execute ISA with various microarchitectures that vary according to the development of new technologies. Therefore, processors may be physically designed to execute a certain instruction set without modifying the ISA.

Simply put, microarchitecture is the purpose-built logical arrangement of the microprocessor's electrical components and data pathways. It facilitates the optimum execution of instructions.

For example, x86-64 is the ISA used by most modern laptop and desktop computers. It is implemented by various microarchitectures, including those designed by Intel and AMD. Software that is compiled for the x86-64 ISA can run on any microarchitecture designed to use the x86-64 instruction set.

Multiple CPU models may be designed for a particular microarchitecture. For this reason, a microarchitecture is sometimes called a "family" or "generation" of CPU. For example, Intel Kaby Lake (7th generation) and Coffee Lake (8th generation) are separate microarchitectures, each with a "family" of compatible CPUs.

System Design / System Architecture

System Architecture defines the high-level structure and the framework of the system, setting the foundation and guidelines for the entire system.

System Architecture focuses on the overall structure and interconnections of a computer system's components. Ensures scalability, reliability, and performance of the entire system and balances hardware and software interactions for optimal functionality.

3. CPU Structure

CPUs are defined by the processor or microprocessor driving them.

- Single-core processor: A single-core processor is a microprocessor with one CPU on its die (the silicon-based material to which chips and microchips are attached). Single-core processors typically run slower than multi-core processors, operate on a single thread and perform the instruction cycle sequence only once at a time. They are best suited to general-purpose computing.
- Multi-core processor: A multi-core processor is split into two or more sections of activity, with each core carrying out instructions as if they were completely distinct computers, although the sections are technically located together on a single chip. For many computer programs, a multi-core processor provides superior, high-performance output.

- Embedded processor: An embedded processor is a microprocessor expressly engineered for use in embedded systems. Embedded systems are small and designed to consume less power and be contained within the processor for immediate access to data. Embedded processors include microprocessors and microcontrollers.
- Dual-core processor: A dual-core processor is a multi-core processor containing two microprocessors that act independently from each other.
- Quad-core processor: A quad-core processor is a multi-core processor that has four microprocessors functioning independently.
- Octa-core: An octa-core processor is a multi-core processor that has eight microprocessors functioning independently.
- Deca-core processor: A deca-core processor is an integrated circuit that has 10 cores on one die or per package.

Types of CPU Architectures

- Complex Instruction Set Computing (CISC)
- Reduced Instruction Set Computing (RISC)
- Very Long Instruction Word (VLIW)
- Explicitly Parallel Instruction Computing (EPIC)
- Stack Architecture
- Dataflow Architecture
- Hybrid Architectures
- Neuromorphic Architecture

4. Flynn's Taxonomy of Computer Architecture

Flynn's taxonomy is a fundamental concept in computer architecture that categorizes systems based on the number of instruction and data streams they can process. Proposed by Michael Flynn in 1966, this taxonomy provides a simple yet effective way to classify computer architectures based on their parallelism capabilities. Flynn's taxonomy is defined as a classification system that categorizes computer architectures into four categories based on the number of instruction streams and data streams they can handle.

Flynn's taxonomy categorizes computer architectures into four categories:

SISD (Single Instruction, Single Data)

SIMD (Single Instruction, Multiple Data)

MISD (Multiple Instruction, Single Data)

MIMD (Multiple Instruction, Multiple Data)

SISD: Single Instruction, Single Data

SISD architectures are characterized by a single processing unit that executes a single instruction stream on a single data stream. This category represents the traditional von Neumann architecture, where a single processor executes instructions sequentially.

The key characteristics of SISD architectures are

- Single processing unit
- Single instruction stream
- Single data stream
- Sequential execution of instructions

Examples of SISD systems include

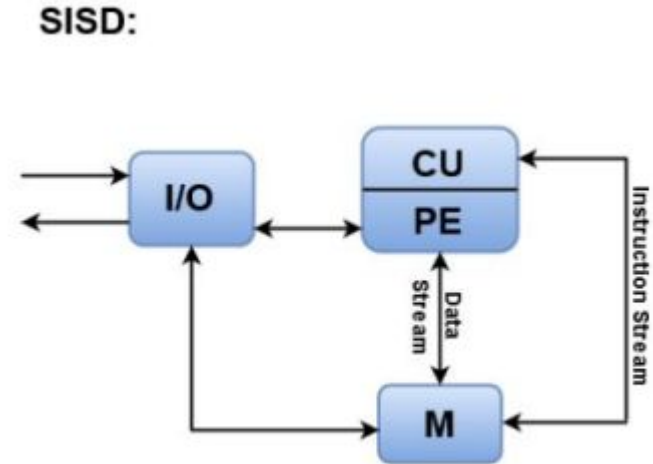
- Traditional desktop computers
- Older mainframe computers
- Many embedded systems

The advantages of SISD architectures include:

- Simple design and implementation
- Easy to program and debug
- Low hardware cost

However, SISD architectures also have some significant disadvantages:

- Limited performance and scalability
- Inefficient for parallel workloads
- Not suitable for applications requiring high throughput



SIMD: Single Instruction, Multiple Data

SIMD architectures are designed to execute a single instruction stream on multiple data streams. This category represents systems that can perform the same operation on multiple data elements simultaneously.

The key characteristics of SIMD architectures are

- Single instruction stream
- Multiple data streams
- Parallel execution of instructions on multiple data elements

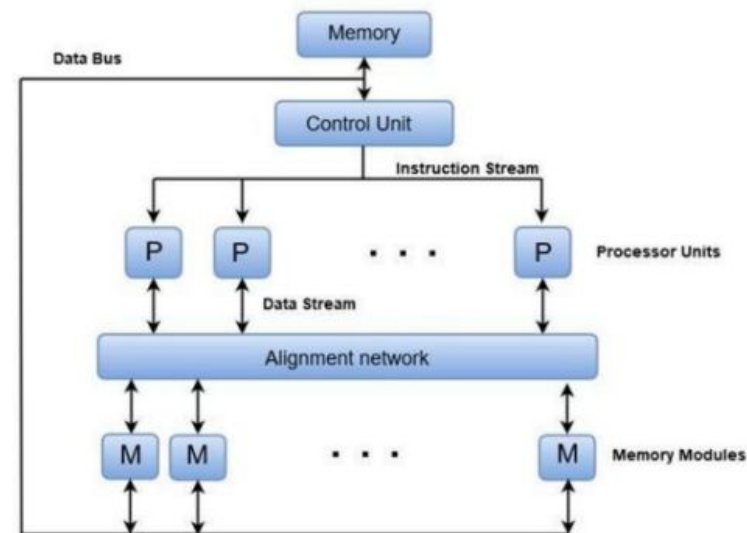
Examples of SIMD systems include

- Graphics Processing Units (GPUs)
- Vector processors
- Some digital signal processing (DSP) architectures

Advantages of SIMD

- **High Performance:** Executes one instruction on multiple data elements simultaneously, making it ideal for data-parallel tasks.
- **Significantly speeds up operations** like vector/matrix computations, image processing, and scientific simulations.
- **Efficiency:** Reduces instruction fetch overhead by applying a single instruction to multiple data points. Optimizes resource utilization by processing multiple data streams in parallel.
- **Scalability:** Scales well for tasks involving large datasets or repetitive operations. Suitable for modern GPUs and vector processors.
- **Energy Efficiency:** Performs more computations per clock cycle, reducing energy consumption for parallelizable tasks.
- **Wide Applications:** Used in graphics rendering, machine learning, signal processing, and scientific computing.

SIMD:



Disadvantages of SIMD

- Limited Flexibility: Only effective for tasks with data-level parallelism. Struggles with irregular or branch-heavy algorithms.
- Complex Programming: Requires specialized programming techniques (e.g., vectorization) to exploit parallelism. Not all algorithms can be easily adapted for SIMD execution.
- Hardware Dependency: Performance gains depend on the availability of SIMD-capable hardware (e.g., GPUs, vector processors). Not all processors support SIMD instructions.
- Data Alignment Issues: Requires data to be aligned in memory for efficient processing, which can complicate memory management.
- Underutilization for Non-Parallel Tasks: Inefficient for sequential or scalar operations, leading to underutilization of resources.

MISD: Multiple Instruction, Single Data

MISD architectures are characterized by multiple processing units that execute multiple instruction streams on a single data stream. This category represents systems that can perform multiple operations on the same data element simultaneously.

The key characteristics of MISD architectures are

- Multiple instruction streams
- Single data stream
- Multiple processing units

Examples of MISD systems are relatively rare, as this architecture is not commonly used in practice. However, some examples include

- Systolic arrays
- Some fault-tolerant systems

Advantages of MISD

- Theoretical Interest: Provides a unique perspective on parallel computing and instruction-level parallelism. Useful for academic research and exploring unconventional architectures.
- Specialized Use Cases: Could potentially be applied in fault-tolerant systems where multiple processors verify or process the same data differently.

Disadvantages of MISD

- Few Practical Implementations: Almost no real-world systems have been built using MISD architecture due to its impracticality.
- Limited Applicability: Not suitable for most real-world computing tasks, which require multiple data streams or shared data processing.
- Complexity: Managing multiple instruction streams on a single data stream is complex and inefficient.

MIMD: Multiple Instruction, Multiple Data

MIMD architectures are designed to execute multiple instruction streams on multiple data streams. This category represents systems that can perform multiple operations on multiple data elements simultaneously.

The key characteristics of MIMD architectures are

- Multiple instruction streams
- Multiple data streams
- Multiple processing units
- Parallel execution of instructions on multiple data elements

Examples of MIMD systems include

- Multiprocessor systems
- Distributed computing systems
- Many modern high-performance computing (HPC) systems

Advantages of MIMD

- High Performance: Enables parallel execution of multiple instructions on multiple data streams, significantly improving performance for complex tasks.
- Flexibility: Supports a wide range of applications, from scientific simulations to real-time processing. Each processor can execute different programs independently, making it versatile.
- Scalability: Can scale to a large number of processors, making it suitable for high-performance computing (HPC) and distributed systems.
- Efficient Resource Utilization: Processors work independently, reducing idle time and maximizing resource usage.
- Distributed Memory Architecture: Each processor has its own local memory, reducing memory contention and improving efficiency. Supports shared memory or message passing for inter-processor communication.
- Fault Tolerance: If one processor fails, others can continue operating, enhancing system reliability.

Disadvantages of MIMD

- Complexity: Designing and programming MIMD systems is complex due to the need for synchronization and communication between processors.
- Communication Overhead: Inter-processor communication (e.g., message passing) can introduce latency and reduce performance.
- Cost: Requires expensive hardware and infrastructure to support multiple processors and memory systems.
- Programming Challenges: Writing parallel programs for MIMD systems requires expertise in parallel programming models (e.g., MPI, OpenMP). Debugging and optimizing parallel code can be difficult.
- Load Balancing: Ensuring an even distribution of work across processors can be challenging, leading to inefficient resource utilization.

Thank You

02.04.2026